

Case Studies – Project 1

Point forecasts of Covid new cases and deaths. Approach using linear models

Deadline: May 20 2026, at 2:10 pm

Submission: Submit your report (together with corresponding R or Python code in a separate file) electronically via moodle.

As mentioned in the course, the data is available at https://github.com/robert-koch-institut/SARS-CoV-2-Infektionen_in_Deutschland

After having started work with your group, in this assignment you will produce forecasts of new Covid cases and deaths in Germany using linear models. Letting y_t denote either the number of cases or the number of deaths at time t , $t \in \{1, 2, \dots, T\}$, your task will consist of producing forecasts of these variables using lagged predictors. You will evaluate the quality of your forecasts in out-of-sample data using the root mean squared forecast error (MSFE).¹

Concretely, you have to accomplish the following tasks for the daily series in the dataset. To do that, you first need to undertake data preparatory steps, which we mentioned in our meeting, but describe more concretely now.

- a) Once you have imported and cleaned the data as in the mock assignment, explore the structure of both series, focusing on time series aspects like serial correlation and variance. For that, you will produce plots of the ACF and PACF. As mentioned in the course, among other things, the seasonal structure of the data at different lags should become evident there and you need to comment on this.
- b) Fit an autoregressive model of order p , $AR(p)$ (i.e. a regression of y_t on $y_{t-1}, \dots, y_{t-p}$); select p using an information criterion. (A compact model representation is given as $(1 - a_1 B - a_2 B^2 - \dots - a_p B^p)y_t = \varepsilon_t$, where the so-called backshift operator B simply lags a variable one period, $By_t = y_{t-1}$, $B^2 y_t = By_{t-1} = y_{t-2}$ etc., and ε_t stands for a sequence of uncorrelated regression disturbances.)
- c) Based on this full-sample autoregressive fit, generate forecasts in the sample and compute the corresponding $MSFE$. (Think why this would not exactly be doable in real life.) Illustrate the forecasts together with the actual load in one plot.
- d) We will use other predictors and not just the autoregressive structure of the variable. One predictor is already given in the dataset, and it is age group. Additionally, you need to find a time series corresponding to average temperature in Germany for the same time frame as the deaths/cases series; the source is up to you. You can add a couple more variables of your liking,

¹Using generic notation, let $\hat{z}_t$ be a forecast of the variable z at time t and let z_t denote the actual value of z at time t , $t = 1, \dots, T$. The mean squared forecasting error over the whole period is then defined as $MSFE_z := \sqrt{\frac{1}{T} \sum_{t=1}^T (z_t - \hat{z}_t)^2}$.

but if you do, make sure you motivate your reasoning for doing so and keep in mind the extra work it requires.

- e) Some practitioners argue that it is better to work with a detrended / deseasoned series. As a next step beyond a pure $AR(p)$ model, estimate a seasonal autoregressive model with an annual lag,

$$\Phi(B^{365})\phi(B)y_t = \varepsilon_t.$$

Here, $\phi(B) = 1 - \phi_1 B - \dots - \phi_p B^p$ is the nonseasonal AR polynomial of order p , which captures short-run dependence across consecutive days, while $\Phi(B^{365}) = 1 - \Phi_1 B^{365} - \dots - \Phi_P B^{365P}$ is the seasonal AR polynomial of order P , which captures recurring annual dependence in daily data. Very briefly, this yields a simple seasonal filtering device: it allows the model to account for systematic yearly variation (for example, a summer–winter pattern) in addition to the short-run dynamics already present in the $AR(p)$ part, which may improve forecasts.

A convenient implementation in R is available in the package `forecast`. In the code below, `p` is the order of the ordinary AR part, `P` is the order of the seasonal AR part, and `365` is the seasonal period because the data are observed daily and the seasonal cycle is annual:

```
fit <- forecast::Arima(  
  y,  
  order = c(p, 0, 0),  
  seasonal = list(order = c(P, 0, 0), period = 365)  
)
```

```
fc <- forecast::forecast(fit, h = 30)
```

Here, `y` is the daily time series, `h = 30` requests 30-step-ahead forecasts, `order = c(p,0,0)` specifies the nonseasonal $AR(p)$ part, and `seasonal = list(order = c(P,0,0), period = 365)` specifies the seasonal AR part with annual lag structure. For example, setting `p = 1` and `P = 1` gives an $AR(1) \times SAR(1)_{365}$ model.